

StepCheck: Sound Static Verification of Deployed AWS Step Functions Workflows

Anonymous Author(s)

Anonymous Institution
anonymous@example.org

Abstract. AWS Step Functions workflows are executable JSON artifacts that run real business processes, yet available validators check only syntax and structural well-formedness: data-flow mistakes, unsafe retries, missing compensations, parallel write races, and unbounded callbacks all deploy and fail only in production. We present STEPCheck, a static verifier for the deployed artifact itself. Its core is a flow-sensitive analysis of the ASL I/O pipeline that reports a missing-field access as an error only when, under the modeled ASL I/O semantics, every execution reaching the read would fail: in the native and declared tiers, every error-level report is a true positive, while inferred premises yield only non-blocking warnings. Seven further checks cover structural, contract, ordering, retry, concurrency, temporal, and compensation properties, graded by the provenance of the semantic facts behind them. On real workflows spanning AWS samples, CNCF examples, independent definitions, and SAM/CloudFormation/CDK deployments, schema validators miss all the targeted semantic classes, and three formal workflow-soundness checkers (Woflan, BPMN Analyzer, BProVe) detect almost none of them while misclassifying 8–29% of valid multi-terminal workflows as unsound. An industrial automotive case study’s lead developer confirmed timeout, data-flow, and concurrency faults, and STEPCheck re-finds independently fixed production bugs. It runs as a drop-in build-time gate with latency below 21 ms.

Keywords: Serverless computing · Workflow orchestration · AWS Step Functions · Static analysis · Data-flow analysis · Saga compensation

1 Introduction

Serverless applications increasingly encode business processes as managed workflows. In AWS Step Functions, the executable artifact is an Amazon States Language (ASL) state machine: JSON for task calls, branches, retries, parallelism, and error handling [5]. These workflows charge cards, reserve inventory, move documents, and coordinate human approvals, so latent workflow bugs become production failures rather than local exceptions. Figure 1 shows one such workflow. It passes AWS’s syntactic validation, yet five latent defect classes survive and a sixth is one edge away. **(1)** `ChargeCard` reads a missing `$.total`; **(2)** reordering a single edge would let it ship before payment; and **(3)** a broad

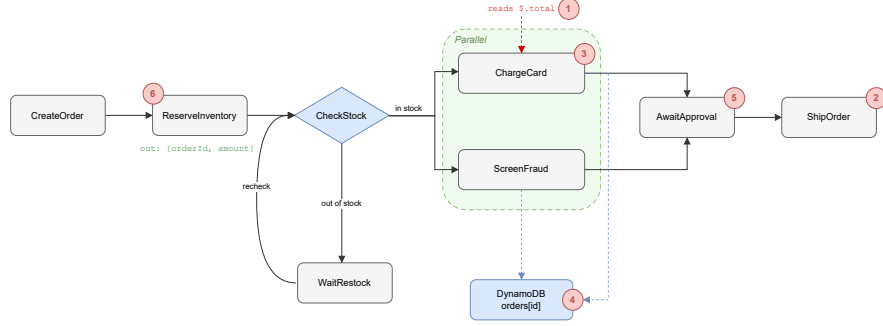


Fig. 1. An order-processing workflow that passes AWS’s syntactic validation yet hides five latent defect classes, with a sixth one edge away; marker **(2)** is a one-edge ship-before-pay mutation, not an existing edge.

retry can double-charge. The same workflow also shows **(4)** two parallel branches updating the same item, **(5)** a callback that can wait forever, and **(6)** a committed reservation without compensation. Existing validators check that the JSON is well-formed, transitions exist, and path expressions are syntactically valid [2,7,10]. They do not check whether a task produces the field its successor reads, whether a retry is safe for a non-idempotent effect, whether two branches race on one resource, or whether a durable effect is compensated after failure. The missing facts are semantic: task contracts, protocol states, idempotency, write footprints, time bounds, and compensators. Formal workflow-soundness verifiers reason about control-flow structure, not ASL’s data movement, retry, or compensation semantics, and flag valid multi-terminal serverless workflows as unsound [27,19,12]. Durable-execution systems enforce recovery at run time [11], not before deployment.

We present STEP CHECK, a static verifier for AWS Step Functions. Its thesis is simple: verify the *deployed artifact* under *layered semantics*. The architectural contribution is a single workflow IR with a *provenance-aware* guarantee model. Facts native to ASL support immediate structural, data-flow, temporal, and concurrency checks that are either sound—every error report a true positive under the modeled semantics (Theorem 1)—or deliberately under-approximating. The business facts ASL omits—contracts, protocol states, idempotency, persistence, compensators—are recovered as provenance-labelled premises from a sidecar, typed DSL, generated metadata, or lightweight inference, and STEP CHECK grades every finding by that provenance: inferred facts produce warnings, declared facts produce errors relative to the declared model, so a finding’s severity encodes the strength of the evidence behind it. Adoption therefore starts on raw ASL and strengthens as teams add semantics. Within this architecture, the core technical result is a flow-sensitive JSON-shape analysis for ASL’s I/O pipeline: it reports a missing-field access *as an error* only when, under the modeled semantics, every execution reaching the read lacks the field (Theorem 1). Seven further passes over the same IR cover structural correctness, typed contracts, typestate ordering, retry safety, Saga compensation, conservative concurrency, and temporal

bounds. In evaluation, STEPCHECK surfaces genuine, deployable defects in real workflows—an industrial automotive case study whose lead developer confirmed timeout, data-flow, and concurrency faults, faults re-discovered in production fix-commit and issue-tracker reports, and findings across independent public repositories—and runs as a drop-in build-time gate (below 21 ms) that blocks semantic defects the deployed AWS validator accepts. Our contributions are:

- A deployed-artifact verification architecture for ASL with native and declared semantic layers.
- A data-flow analysis for missing JSON fields whose error-level reports are free of false positives under the modeled JSONPath/JSONata semantics, plus workflow checks for retry, ordering, concurrency, time, and compensation.
- A Rust implementation with raw ASL, typed-DSL, CNCF Serverless Workflow, and CloudFormation/SAM/CDK front ends.
- Empirical evidence that these defect classes occur in deployed workflows and evade both industrial validators and formal soundness verifiers—whose single-sink soundness conversely misclassifies 8–29% of valid multi-terminal workflows—via mutation studies, a developer-confirmed industrial case study, re-found production faults, cross-format artifacts, and a sub-21 ms CI gate.

2 Background and Problem Setting

ASL exposes enough structure for native checks, but hides the business facts needed for deeper workflow verification. This section fixes the small slice of ASL semantics used by the analyses, then states the omitted facts that leave the harder defects latent. A Step Functions workflow is a state machine with a **StartAt** state and named states of eight kinds: **Task**, **Choice**, **Parallel**, **Map**, **Wait**, **Pass**, **Succeed**, and **Fail**. A single JSON document is threaded through the graph, and each state may reshape it through a fixed five-stage I/O pipeline (**InputPath** through **OutputPath**) whose per-stage semantics Section 3.2 models directly. Reads against the document are explicit in the syntax. A key ending in **.\$** reads a JSONPath from the current document (e.g. a **"Payload.\$": "\$.detail.customer"** constructor reads **\$.detail.customer**), and the runtime fails the reference when that path is absent. ASL additionally provides workflow variables and JSONata expressions: **Assign** stores **\$var** values for later states, and JSONata fields read **\$states.input**, **\$states.result**, and workflow variables. Fault handling is likewise explicit: **Retry** re-executes a state on matching errors, **Catch** routes an error to a handler, and **TimeoutSeconds/HeartbeatSeconds** bound long-running tasks and callbacks. What the artifact records, an analysis can check; the question is how far that record reaches.

The intended protocol in Figure 1 is *created* → *reserved* → *paid* → *shipped*. What ASL records already decides three of the six defects: (1) follows from the I/O pipeline alone, and the racing writes (4) and unbounded callback (5) from statically named resources and absent bounds—natively but conservatively. The other three depend on facts ASL has no syntax for: a task

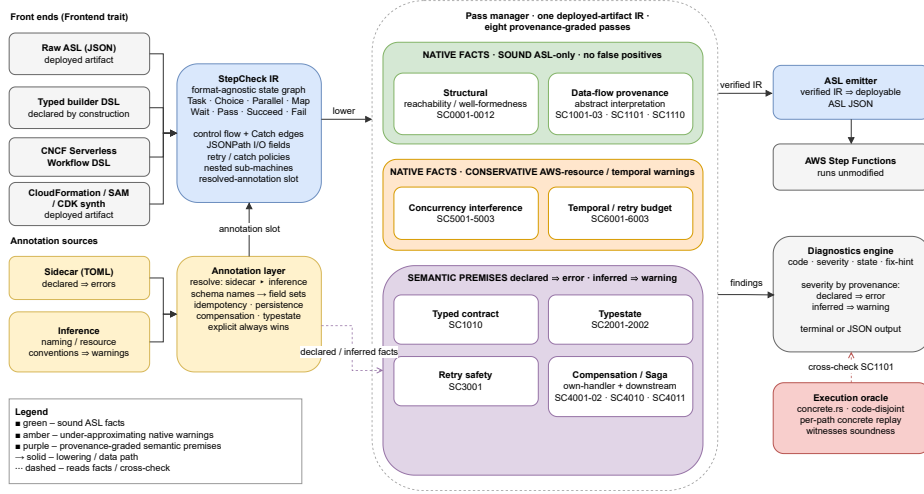


Fig. 2. Verification pipeline: all front ends lower to one deployed-artifact IR shared by the eight passes, rather than running independent lints.

contract $T : A \rightarrow B$, without which reordering (2) is just another valid edge; a typestate edge $s_{in}^T \rightarrow s_{out}^T$ in a business protocol δ ; idempotency of external effects, without which the double-charging retry (3) looks safe; and persistence plus a compensator, without which the uncompensated reservation (6) looks like any other task. Because deployed validators inspect only syntax and structural well-formedness, all six defects deploy: each is valid JSON over a valid graph.

3 Verification Framework

A verifier for these latent defects must (i) check the deployed ASL artifact *in place*, at build time, without a new runtime; (ii) be *sound where native*, when a property follows from ASL alone, report only definite faults, so a CI gate can block without triage; (iii) accept the omitted facts as *semantic premises* from sidecars, DSLs, or inference, grading severity by provenance; and (iv) cover all six defect classes over *one representation* rather than as independent lints. STEPCheck separates the artifact being checked from the semantic facts available about it: a shared, format-agnostic IR records the control flow and data movement that ASL exposes, and a layered annotation slot records the contracts, business states, idempotency, persistence, and compensators that ASL omits. Optional name/resource inference gives existing repositories a non-blocking start, and severity tracks provenance (native and declared violations error, inferred facts warn).

3.1 Architecture

Every front end lowers to the same workflow graph (Figure 2); every analysis is a pass over it; and the optional emitter lowers verified DSL-authored workflows back to deployable ASL. Raw ASL is the primary input, parsed tolerantly through

Table 1. Diagnostic codes by class, severity, and soundness tier: *sound* (no false positives, Thm. 1), *conservative* (abstains rather than over-reports), *declared* (exact given annotations), *advisory* (inferred, non-blocking).

Code(s)	Class	Severity	Tier	Required premise
SC0001–SC0010	structural well-formedness	error/warn	sound	—
SC0011/SC0012	sidecar consistency	error	declared	supplied sidecar
SC1001–SC1003	native data binding	error/warn	sound	valid ASL grammar
SC1101	data-flow field provenance	error/warn	sound	modeled fragment (else $\top$); warn = inferred contract
SC1110	Choice-guard provenance	warning	conservative	modeled fragment (else $\top$)
SC1010/SC2001/SC2002	contracts / typestate	error	declared	declared schemas/protocol
SC5001–SC5003	concurrency interference	error/warn	conservative	static resource/child key
SC6001–SC6003	temporal bounds	error/warn	conservative	declared bounds
SC4002/SC4011	declared saga consistency	error	declared	declared compensator
SC3001/SC4001/SC4010	retry / compensation	error/warn	advisory/declared	inferred or declared effects

generic JSON, so intrinsics, context references, SAM placeholders, dialect variants, and JSONata states never cause hard parse failures. The typed DSL emits graph and annotations by construction, and the CNCF front end exercises the same extension point. The pass families are summarized in Table 1. Native structural and data-flow checks need only the ASL artifact. The *conservative* concurrency and temporal checks report only provable conflicts and may miss others—the opposite polarity to the over-approximating, sound data-flow analysis of Section 3.2—and therefore warn, except the definite misconfiguration SC6003. Semantic checks verify consistency with supplied premises; they do not prove those premises true.

Structural and annotation hygiene (SC0001–0012) is the base well-formedness layer: it flags undefined targets, unreachable or dead-end states, non-exhaustive **Choice**, and stale sidecar references. The remaining data-flow, concurrency, temporal, and declared-tier passes are detailed in Sections 3.2–3.4, and the technical report [22] tabulates the full pass-by-pass claim strengths. The IR itself is a recursive graph of named states with a designated start. Each state records its ASL kind, normal successors, **Catch** edges, the five-stage I/O pipeline, retry and timeout controls, JSONPath-vs-JSONata mode, nested **Map/Parallel** sub-machines, and the annotation slot. This is enough for every pass; surface syntax stays in the front ends.

3.2 Native Data-Flow Verification

The native layer is a sound bug finder, not a complete verifier. If STEPCheck reports SC1101 as an error, then under the modeled semantics of Theorem 1, the referenced field is absent on every execution reaching that read. In inferred-contract mode, SC1101 downgrades to a warning meaning not guaranteed by the contract premise; the definite-absence reading and Theorem 1 apply to the native and declared closed-world tiers. If it is silent, the field may still be absent for some inputs hidden behind $\top$. This is the right direction for a raw-artifact checker facing arbitrary input and opaque task results.

The analysis tracks *document and variable shapes*: either $\top$ (opaque: any field may be present) or a finite record of fields that may occur. Joins union keys, with $\top$ absorbing; a path is definitely absent when record descent reaches a missing

key before $\top$. The transfer functions mirror the five-stage ASL I/O pipeline and the **Assign/Catch** semantics: path stages re-root the shape, constructors build closed records while checking $.\$$ reads, unknown task and sub-workflow results lift to $\top$ unless a declared output schema or a fixed AWS service envelope pins their keys, and **ResultPath** merges results into the raw input (full per-construct cases in [22]). For JSONata, **STEPCHECK** models direct $\$states.*$ and $\$var$ references plus object-valued **Output/Assign/Arguments** fragments; unsupported expressions project to $\top$ and never produce **SC1101**. A forward iteration joins shapes along normal and **Catch** edges. Cycles can re-embed a sub-document under a **ResultPath** and grow record depth without changing the key set, so **STEPCHECK** applies depth- k widening after every propagation step: any record nested deeper than k (the implementation uses $k = 12$) is replaced by $\top$. Widening is conservative—it can hide a bug behind opacity, but it cannot manufacture a definite absence. With a finite key universe and bounded record depth, the lattice has finite height, so the iteration terminates; a defensive round bound lifts all entries to $\top$ rather than ever reporting from an under-approximation (formula and telemetry in [22]). In all experiments fixpoints converge within 7 rounds. The restock loop in Figure 1 illustrates why a fixpoint matters. **CheckStock** has one incoming shape from **ReserveInventory** and one from the **WaitRestock** back-edge; joining them keeps any loop-produced field alive, so **SC1101** fires on the downstream **ChargeCard** read only because *no* iteration produces **total**.

Theorem 1 (Sound bug reports for missing-field references). *Under the modeled JSONPath/JSONata document-and-variable semantics, if STEPCHECK reports SC1101 as an error for a path p at state s , then every concrete execution reaching s lacks p in the document or variable selected by that read site, so the ASL runtime fails that reference. In the native tier this holds for arbitrary execution input; in the typed tier it holds relative to the closed-world reading of declared input schemas and, when result-shape modeling is enabled, declared task-output schemas and fixed AWS service-result envelopes. Expressions outside the modeled fragment lift to $\top$ and are therefore silent.*

Proof (Proof sketch). Let γ map each abstract shape to the concrete JSON values it may denote, with $\top$ denoting all values and records exactly their listed keys. Every transfer function over-approximates the corresponding ASL operation under γ ; joins are least upper bounds and depth- k widening only replaces sub-records by $\top$, hence is extensive, so the forward iteration computes a post-fixpoint over-approximating every concrete environment reaching each state, including all loop iterations. If lookup of p at the read site reaches a missing key before $\top$, no value in the concretization contains p , so the modeled ASL read fails on every execution reaching s ; expressions outside the modeled fragment reach $\top$ and cannot trigger the premise. The full proof, with per-construct transfer-function cases, is in [22].

The guarantee is relative to the modeled fragment, not verified against AWS’s implementation; unmodeled expressions lift to $\top$ and stay silent by construction, the execution oracle finds zero counterexamples across all reports, and differential validation of the model is future work. **SC1110** applies the same definite-absence

judgment to **Choice** guards but stays a warning outside Theorem 1: rule reachability depends on earlier guards such as **IsPresent**, so it marks a provably dead or risky branch rather than an unconditional runtime fault.

3.3 Concurrency and Temporal Analyses

The concurrency and temporal passes are conservative engineering checks. **Parallel** branches and concurrent **Map** iterations receive document copies, so only external resources can race. **STEPCHECK** records statically known resource keys (**TableName**, **QueueUrl**, **Bucket**, and other named targets; full list in [22]). It emits **SC5001** only for provably overwriting operations—**PutItem**/**PutObject** and statically named **states:startExecution** collisions—leaving keyed, append-only, and compute-only operations silent. The temporal pass checks resilience knobs the platform does not cross-check. **SC6001** warns when a callback or Activity task lacks a timeout or heartbeat and can wait to the platform maximum. **SC6002** warns when a retry budget can exceed the machine timeout. **SC6003** errors on the definite misconfiguration **HeartbeatSeconds** $\geq$ **TimeoutSeconds**.

3.4 Semantic Verification

The semantic layer consumes task facts ASL does not contain: a contract $T : A \rightarrow B$ (fields consumed and produced), a typestate transition $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ in protocol δ , idempotency (whether a retry repeats an external effect), and persistence with an optional compensator. These are premises (Section 3.5); **STEPCHECK** checks the workflow relative to them.

Proposition 1 (Declared-tier consistency). *For a fully declared workflow W , **SC1010**/**SC2001**/**SC2002** emit no finding iff every normal-flow task adjacency $T_1 \rightarrow T_2$ satisfies $\text{in}(T_2) \subseteq \text{out}(T_1)$ and $s_{\text{out}}^{T_1} = s_{\text{in}}^{T_2}$, and every task transition belongs to the declared protocol δ . On the series-parallel fragment, this edge condition is equivalent to a standard field/typestate typing judgment. Completeness is relative to the supplied annotations.*

Here $\text{out}(T_1)$ is T_1 ’s declared output set—including fields it forwards, not only those it computes—and $\text{in}(T_2)$ is T_2 ’s declared input set; under this closed-world convention the edge-local check does not false-positive on a field merged upstream and passed through. The checks are edge-local: a ship-before-pay reordering in the order workflow yields typestate mismatches plus field-contract misses. Retry-safety is likewise premise-relative: a broad **States.ALL**/**States.TaskFailed** retry of a task declared non-idempotent is **SC3001**, while retries narrowed to specific transient classes are left alone. Per-task checks (**SC4001**/**SC4010**) can see that a persistent task has no compensating path, but they miss a later state failing after the effect has committed. For a declared persistent task P with compensator C , **SC4011** scans the post-commit region reachable from P before C and asks whether every fallible state’s generic failure routes through a broad **Catch** to C , and whether no normal path reaches **Fail** while bypassing C . The declared Saga model assumes a single post-commit fault, feasible normal paths, a unique generic catcher for **States.TaskFailed**, and linear handlers whose reachability to C means execution of C .

Theorem 2 (Relative soundness and completeness of the compensation check). *For a workflow with declared persistence and compensators, under the declared Saga model, SC4011 emits no finding for persistent task P with compensator C iff every modeled post-commit failure after P executes C . Thus SC4011 is sound and complete relative to the declared model.*

Outside these assumptions, SC4011 remains a reachability check but is no longer sound or complete. The model-independent SC4001/SC4010 checks still flag any persistent effect with no compensating path. Coverage therefore degrades to the per-task tier rather than going silent; the built-in and corpus sagas satisfy the assumptions.

3.5 Recovering Workflow Semantics

Declared facts enter through a sidecar for existing ASL or the typed DSL for greenfield workflows; once declared they support error-level findings, and where absent the declared-tier checks are silent. Concrete sidecar syntax is deferred to the technical report [22]; the analyses use only the declared idempotency, persistence, compensation, and typestate facts. The order saga’s complete sidecar is 68 lines of TOML—about eight per-task lines over four tasks, plus shared schema and protocol blocks—and the typed DSL emits the same facts by construction. Optional inference gives existing repositories a zero-effort warning mode: read-like names imply idempotent, write-like non-idempotent, resource-acquiring persistent, and notifications non-idempotent but not persistent; with no signal it abstains, and explicit annotations override it. Stale sidecar and schema references are caught syntactically (SC0011/SC0012). The residual risk, semantic drift between a declared premise and the deployed Lambda, is outside STEP CHECK’s scope; generating the sidecar from code or IaC keeps declared facts in sync.

4 Implementation

STEP CHECK is a single dependency-light Rust command-line tool: four front ends lower an input format to one shared IR, eight passes analyse that IR, and an optional emitter lowers verified DSL-authored workflows back to deployable ASL. It ships as one self-contained binary, packaged for `crates.io` and `npm` (packaging, command flags, and exit-status details in [22]). After front-end lowering, annotation resolution runs once: it validates sidecar references, materialises declared schema fields, and fills optional inferred effect facts. The default pass order is structural $\rightarrow$ contract $\rightarrow$ typestate $\rightarrow$ data-flow $\rightarrow$ concurrency $\rightarrow$ temporal $\rightarrow$ retry $\rightarrow$ Saga compensation. Each pass is a pure reader of the resolved IR and appends only diagnostics, so later passes never depend on facts produced by earlier ones; the sequential order is for deterministic reporting, not for mutation. A serverless team rarely commits a standalone ASL file: the state machine lives inside a CloudFormation or SAM template, or is generated by CDK. STEP CHECK’s CloudFormation front end reads these deployment artifacts—including CDK-synthesized templates—directly, resolving the embedded template intrinsics and linking sibling state machines so cross-machine checks compose, and lowers each

to the shared IR with no manual extraction. This makes the build-time gate a drop-in check on the artifact a team actually ships. STEPCheck exposes a single build-gate entry point, `stepcheck check` on the deployed artifact, which emits provenance-graded diagnostics, each an error or warning per Table 1. The *recommended CI policy* fails a build on sound native and declared-tier errors while leaving *inferred warnings* non-blocking; teams can later configure warning-level findings to block once annotations mature. The implementation is validated by per-pass unit and regression tests, a seeded-mutation golden harness, and two code-disjoint self-checkers for SC1101 (Section 5; per-pass test breakdown in [22]).

5 Evaluation

We organize the evaluation around four questions plus one case study. RQ1 asks whether STEPCheck detects the defect classes that schema validators and workflow-soundness verifiers miss. RQ2 isolates the central data-flow claim: when SC1101 reports a missing field, is the report sound, and how much of real ASL falls in the modeled fragment? RQ3 tests whether the findings generalize beyond the core AWS sample corpus to unmodified workflows, real fixes, other formats, independent repositories, and deployment artifacts. RQ4 measures whether STEPCheck is cheap enough to run as a build-time gate. The automotive case study then tests the checks and annotation workflow on a private production rewrite. The evaluation corpus is public and deployment-shaped. Its core is **193** AWS sample workflows [6], mined exhaustively from every parseable ASL definition and spanning the main ASL constructs, JSONata-mode, Distributed Map, and nine AWS services plus six industrial-topology `aws-samples` patterns including AWS Serverless Airline Booking `ProcessBooking`, a canonical serverless benchmark [1]. For external validity (§ 5.3) we add `serverless-patterns` and AWS Solutions Library deployment artifacts [6,4], CNCF Serverless Workflow examples, and independent ASL from public repositories; these public artifacts are reproducible stand-ins for proprietary definitions, which we complement with a private, anonymized automotive case study (§ 5.5). All experiments use a Rust 1.96 release build on a Windows 11 laptop with an Intel Core i7-8550U CPU (4 cores/8 logical processors) and 16 GB RAM.

5.1 RQ1: Detection power and baselines

STEPCheck detects the semantic defect classes existing tools miss. On the 193 AWS sample workflows [6] we inject one defect per applicable workflow crediting a detection only when re-verification emits a fresh diagnostic naming the injected fault. Across 616 mutants in six defect classes STEPCheck detects every one, with exact 95% confidence intervals per class (Table 2). The schema validators—`statelint` [7], `asl-validator` [10], and AWS’s `ValidateStateMachineDefinition` [2]—catch the schema-level dangling transitions and broken bindings but none of the four semantic classes. Because the mutation operators are ours, we also run a matched hard-mutant suite—**688**

Table 2. Injected-defect detection over 616 mutants (per-class recall): STEPCheck versus three schema validators and three formal verifiers. A mutant counts as detected only when a tool emits a fresh problem naming the injected fault; STEPCheck alone catches the semantic classes.

Defect class	Check	n	STEPCheck (CI)	Schema validators			Formal verifiers		
				statelint	asl-val.	AWS	Woflan	BPMN-A.	BProVe
Dangling transition	SC0002	166	1.00 [.98,1]	1.00	0.94	1.00	0.95	0.87	0.69
Broken data binding	SC1003	141	1.00 [.97,1]	0.99	0.94	0.96	0.00	0.00	0.00
Unsafe retry	SC3001	103	1.00 [.96,1]	0	0	0	0.00	0.00	0.00
Missing compensation	SC4001	19	1.00 [.82,1]	0	0	0	0.37	0.37	0.11
Concurrency interference	SC5001	21	1.00 [.84,1]	0	0	0	0.00	0.00	0.05
Temporal heartbeat/timeout	SC6003	166	1.00 [.98,1]	0	0	0	0.00	0.00	0.00
Formal-verifier misclassification of <i>valid</i> workflows (of 193)							16 (8.3%)	16 (8.3%)	56 (29%)

mutants that keep each semantic fault but shift it just past the analysis’s proof ($\top$) boundary. Aggregate recall is **0.24** (162 of 688): the data-flow, contract, concurrency, and temporal checks fall to 0 once the fault crosses their $\top$ boundary, retry and compensation degrade to 0.68/0.79, and the schema-level structural check holds at 1.00 (per-class breakdown in [22]). Every miss is a premise the analysis soundly lifts to $\top$, not a false alarm: a sound analysis (Theorem 1) declines to report what it cannot prove, so these expected drops mark completeness boundaries—not unsoundness—and show the checks are not tuned to our operators. We also run three formal workflow-soundness verifiers locally—Woflan [27], BPMN Analyzer 2.0 [19], and BProVe [12]—encoding each ASL workflow into a BPMN workflow net and crediting a verifier only on a fresh soundness violation (Table 2). They express only the single control-flow class, dangling transitions (SC0002), catching it at Woflan 95%, BPMN Analyzer 87%, and BProVe 69%. The four native classes that do not encode as workflow-net soundness—data binding, retry safety, field flow, and temporal bounds—are silent for all three formal verifiers, while compensation and concurrency are caught only when the injected fault also manifests as a workflow-net soundness violation: at most 37% for compensation and 5% for concurrency, far below STEPCheck’s 1.00 on both. They also misclassify valid workflows—Woflan and BPMN Analyzer each flag 16 of 193 (8.3%) and BProVe 56 of 193 (29%)—because ASL’s multi-terminal termination is not a sound workflow net under the single-sink definition these tools implement. Our encoding deliberately preserves the distinct **Succeed/Fail** terminals rather than merging them into a synthetic sink, since merging them would mask the failure-path structure the compensation analysis targets; the rejections reflect a definitional gap (§6); StepCheck’s Asl-native model accepts all 193.

5.2 RQ2: Data-flow soundness and coverage

The flagship sound check SC1101 (data flow) is measured separately from the six classes above: as a no-false-positive check it fires zero times on the clean corpus, so we inject a missing-field read into each of the **126 of 193** workflows that contain a document-field read the mutator can turn into a missing read. With output-shape modeling STEPCheck detects 88 (70%); the remaining 38

injected reads sit behind constructs the analysis soundly lifts to $\top$ —opaque task results or unmodeled selectors—so silence there is the completeness boundary, not a missed proof. A proof-certificate checker certifies all 88 reports by checking 1,146 finite widened-fixpoint postcondition obligations, and a concrete execution oracle finds zero present-field counterexamples (cf. [21]). The check is practically relevant: 92% of the corpus’s document-field references fall in the fragment it models exactly, the rest lifting soundly to $\top$.

5.3 RQ3: Real-world evidence and external validity

On the same 193 unmodified AWS sample workflows [6], an inference-mode run emits 226 diagnostics: 181 advisory inferred-tier warnings (SC3001/SC4001/SC4010) and 45 native-tier findings across four codes, dominated by 33 unbounded callbacks (SC6001; per-code breakdown in [22]). We adjudicate all 45: a code-disjoint verifier re-checks each finding’s structural predicate against its ASL source and confirms every one. We do not treat the **181** inferred-tier warnings as adjudicated defects here; the technical report [22] separately calibrates the name/resource inference that feeds them against gold task labels. STEPCheck also re-discovers real defects in deployed artifacts. In **39** mined fix-commit pairs, a diagnostic appears only on the buggy revision in **7**; adjudication confirms **6** genuine in-scope defects across data-flow, dead-branch (including an SC1110 unreachable subscriber branch), unbounded-callback, and structural classes and rejects the 7th as an artifact—its finding vanished only because the fix deleted the flagged task, not because the defect was resolved—while the other **32** pairs are out-of-scope value, feature, or refactoring edits. In **three** user-filed issues—each quoting the verbatim runtime error STEPCheck’s sound tier prevents—STEPCheck flags all three genuine SC1101 defects; on **two** further standalone definitions mined from independent repositories it flags one non-idempotent SQS send retried on `States.TaskFailed` (SC3001) and correctly passes one sound saga. Repository, commit, and issue provenance are in the technical report [22]; these results are existence evidence, not a prevalence estimate. Across a different workflow format, the front end lowers **66** CNCF Serverless Workflow examples to the same IR. The structural, tpestate, retry, compensation, concurrency, and temporal checks then apply unchanged, and the mutation study catches every injected control-flow defect. Field-flow generalizes less: only **12 of 66** examples declare records the sound SC1101/SC1110 checks can model. The other **54** are schema-free, so those checks stay silent rather than claim coverage. Where CNCF declares JSON Schemas, SC1010 enforces them exactly. To test for overfitting, we ran STEPCheck on **95** further ASL definitions from **16** public repositories. It produced representative findings—a flow-sensitive missing field, a shared DynamoDB write across `Parallel` branches, and a declared-tier uncompensated payment effect—on workflows we neither authored nor curated [22]. Finally, the CloudFormation front end reads the artifacts teams actually ship. Running STEPCheck on the `aws-samples/serverless-patterns` SAM templates [6], with no manual extraction, covers **55** workflows and flags **13**. Two are sound-tier:

a missing `Default` (SC0007) and an unbounded callback (SC6001). On the enterprise **AWS Solutions Library** [4] (**26** machines, partly CDK-generated) it flags **14**, alongside several clean true negatives. Re-running the gate on six CDK-synth templates catches every applicable mutation, including the semantic ones `asl-validator` silently accepts. This moves the corpus from curated ASL to the deployment forms verification must meet.

5.4 RQ4: Build-time cost and deployability

We evaluate both the build-time gate and verification cost on six industrial-topology workflows totaling 65 recursive states, including four `aws-samples` Sagas [6], a Map-based ETL aggregator, and `SAB ProcessBooking`. As a gate, `STEPCHECK` runs at CI cost: we inject 0/1/5 defects and, per level, compare the build-fail catch-rate against the local schema validator. On the clean set the validator accepts 5 of 6 workflows; its lone rejection is the JSONata-mode ETL workflow, a validator-version lag on ASL’s newer `Assign/Arguments/Output` fields rather than a workflow defect. At a single injected *semantic* defect the gate fails the build on every workflow with an applicable mutation site, while the validator flags *none*: its only rejection is that same JSONata ETL it already misreads, not the injected defect, which would therefore deploy. At five defects the gate again catches every workflow with applicable mutation sites, while the validator flags only co-injected *syntactic* faults, never a semantic one. Gate latency stays below **21** ms at p95 (including process start): on six CDK-synthesized AWS Solutions deployment templates, the same end-to-end gate costs p50/p95 **15.7/20.9** ms. Verification is cheap: a workflow checks in a median $\approx 25 \mu\text{s}$ (max ≈ 1.1 ms), adding no runtime component since it runs before deployment. The widened fixpoint converges on every committed input ($k = 12$ depth; 262 definitions: 193 AWS samples, 66 CNCF examples, and 3 issue-mined workflows; maximum 7 rounds). Scale is not a concern either: a synthetic **10,000**-state workflow verifies in ≈ 31 ms, well under a one-second CI budget. Finally, the emitter round-trips the 193-workflow corpus with exact ASL-object equality for 192 of 193, and `STEPCHECK`-verified definitions deploy unmodified and complete 100/100 live runs across Express, Standard, and callback execution models [22].

5.5 Automotive case study

We also inspected an anonymized automotive packaging-service module: part of an ongoing rewrite of a legacy production system into serverless services, built entirely in CDK by a cross-country team of 12 engineers with uneven Step Functions experience. Three Express workflows—`Borrowed`, `Universal`, and `ValidateSupplier`, with 15, 14, and 5 recursive ASL states—encode packaging business flows: request validation, permission and supplier checks, persistence, and event publication. An initial inference-mode run of `STEPCHECK`—before any annotations, so contract-dependent findings are advisory and outside Theorem 1—reported **40 findings**: **25** retry/timeout mismatches (SC6002; Lambda tasks retrying up to 126 s against declared 10–20 s machine timeouts), **10** data-flow

warnings (SC1101), and **5** concurrency warnings (SC5001). The team then authored a declared-tier sidecar for durable effects and data contracts; re-checking promoted the 5 concurrency and 7 of the 10 data-flow warnings to *errors*, confirming parallel writes to the same durable resource and fields missing on some paths. The remaining 3 stayed inferred-tier warnings—*not guaranteed by the contract premise*, per Section 3.2—until the corrected schema declared those fields optional; since the reads are guarded by presence checks, the declared-tier re-run confirmed them consistent and the warnings cleared, the expected outcome for handled-optional fields rather than a silenced defect. Both severity tiers thus behaved as designed: no error-level SC1101 was refuted by the team, upholding Theorem 1’s guarantee. A lead developer confirmed the **12 errors** (SC5001, SC1101) and **25 timeout warnings** as genuine, deployable defects; each was subsequently fixed. The most critical: in **Universal**, a **Map** state’s **ItemsPath** read `$.detail.items`, absent on error-handling paths where `detail` is an error object; the failed path read would pass `null` to the **Map** iterator, causing a **States.Runtime** error—squarely in SC1101’s scope.

6 Related Work

Runtime orchestration and durable execution. Triggerflow rethinks how serverless workflows run [8], while durable-execution runtimes—Azure Durable Functions [11], Temporal [25], and Beldi’s transactional guarantees for stateful serverless workflows [28]—enforce recovery and transactions *during* execution. STEP CHECK is complementary: it statically checks the deployed Step Functions artifact *before* deployment with no new runtime, pursuing the same failure-atomicity goal from the opposite end.

Formal verification at other layers. AWS uses TLA^+ to model-check service designs [20], and Zelkova brings SMT reasoning to IAM access policies [9]. These verify an abstract model, the platform, or a different artifact; STEP CHECK checks the executable ASL definition with linear passes suitable for CI. Woflan [27], BPMN Analyzer [19], and BProVe [12] verify control-flow soundness over Petri-net or BPMN models and do so effectively on that class (Table 2); but they analyse a control-flow abstraction, not the deployed ASL artifact, so data flow, retry, idempotency, and temporal bounds fall outside their property language, and their single-sink criterion predates ASL’s multi-terminal termination, cf. relaxed soundness [14]. Closest to our retry-safety check, Flux verifies idempotence of stateful serverless functions [15] and could *supply* the per-task idempotency premises STEP CHECK’s SC3001 consumes.

Behavioural types and Sagas. Typestate and session types provide the basis for protocol conformance [23,17], and the Saga pattern is the standard compensation model [16]; STEP CHECK adapts both to serverless task graphs as edge-local declared checks and a declared-model reachability check.

Data-flow and cloud validators. Existing Step Functions validators check schema and path syntax, but not field provenance [7,10,2,5]. AWS’s **TESTSTATE** API [3] complements them dynamically, executing states against author-supplied

inputs and mocks; its coverage is thus bounded by the tests written. STEPCheck is the static dual: with no inputs or mocks, its sound verdict quantifies over *every* execution—including paths no test exercises, such as the unreachable branch behind the rediscovered SC1110 bug. Data-flow anomalies have been studied for BPMN and BPEL [24,26], but there data is abstracted to named elements in a hand-built model; STEPCheck tracks nested JSON document shapes through ASL’s I/O pipeline as an abstract interpretation [13]. Serverless formal foundations motivate the setting but do not provide static workflow verification [18].

7 Conclusion

STEPCheck shows that useful static verification can run directly on the workflow artifacts developers already deploy: raw ASL, generated templates, and cross-format definitions. Its native data-flow analysis reports only definite missing-field failures—free of false positives relative to the modeled I/O semantics, with every error report machine-certified against that model—while layered annotations turn retry, typestate, and Saga assumptions into provenance-graded diagnostics. The central lesson from the evaluation is quantitative: classical workflow-net soundness misclassifies 8–29% of valid workflows—a definitional mismatch with ASL’s multi-terminal model—while its vocabulary omits the production bug classes—data flow, retry, concurrency, and temporal—that STEPCheck catches at the artifact level, cheaply enough for a build gate. Serverless orchestration need not wait until execution to expose data, ordering, retry, timing, and compensation mistakes, provided the verifier makes explicit which facts are native, declared, or inferred.

Artifact availability. An anonymized artifact—STEPCheck, corpus, scripts, and reproduction instructions—plus a companion technical report with the full formal development is at <https://anonymous.4open.science/r/StepCheck> [22].

References

1. Amazon Web Services: AWS Serverless Airline Booking. <https://github.com/aws-samples/aws-serverless-airline-booking> (2019)
2. Amazon Web Services: ValidateStateMachineDefinition — AWS Step Functions API reference. <https://docs.aws.amazon.com/step-functions/> (2024)
3. Amazon Web Services: TestState — AWS step functions developer guide. https://docs.aws.amazon.com/step-functions/latest/apireference/API_TestState.html (2025)
4. Amazon Web Services: AWS Solutions Library. <https://aws.amazon.com/solutions/> (2026)
5. Amazon Web Services: AWS step functions developer guide (amazon states language). <https://docs.aws.amazon.com/step-functions/> (2026)
6. Amazon Web Services: Serverless Patterns Collection. <https://github.com/aws-samples/serverless-patterns> (2026)
7. Amazon Web Services Labs: statelint: A validator for Amazon states language documents. <https://github.com/aws-labs/statelint> (2024)

8. Arjona, A., García López, P., Sampé, J., Slominski, A., Villard, L.: Triggerflow: Trigger-based orchestration of serverless workflows. *Future Gen. Comp. Syst.* (2021)
9. Backes, J., Bolignano, P., Cook, B., Dodge, C., Gacek, A., Luckow, K., Rungta, N., Tkachuk, O., Varming, C.: Semantic-based automated reasoning for AWS access policies using SMT. In: *FMCAD*. pp. 1–9 (2018)
10. Bougère, C.: asl-validator: A JSON-schema validator for Amazon states language documents. <https://github.com/ChristopheBougere/asl-validator> (2024)
11. Burekhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C.S.: Durable functions: Semantics for stateful serverless. *OOPSLA* **5** (2021)
12. Corradini, F., Fornari, F., Polini, A., Re, B., Tiezzi, F., Vandin, A.: A formal approach for the analysis of BPMN collaboration models. *J.Sys. Softw.* **180** (2021)
13. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approx. of fixpoints. In: *POPL* (1977)
14. Dehnert, J., Rittgen, P.: Relaxed soundness of business processes. In: *CAiSE* (2001)
15. Ding, H., Wang, Z., Shen, Z., Chen, R., Chen, H.: Automated verification of idempotence for stateful serverless applications. In: *OSDI* (2023)
16. Garcia-Molina, H., Salem, K.: Sagas. In: *ACM SIGMOD ICMD*. pp. 249–259 (1987)
17. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: *POPL*. pp. 273–284 (2008)
18. Jangda, A., Pinckney, D., Brun, Y., Guha, A.: Formal foundations of serverless computing. *OOPSLA* **3**, 1–26 (2019)
19. Kräuter, T., Stünkel, P., Rutle, A., Lamo, Y., König, H.: BPMN Analyzer 2.0: Instantaneous, comprehensible, and fixable control flow analysis for realistic BPMN models. In: *BPM* (2024)
20. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M.: How Amazon web services uses formal methods. *ACM Communications* (2015)
21. Pnueli, A., Siegel, M., Singerman, E.: Translation validation. In: *TACAS* (1998)
22. StepCheck: Repository. <https://anonymous.4open.science/r/StepCheck>, (2026)
23. Strom, R.E., Yemini, S.: Typestate: A programming language concept for enhancing software reliability. *IEEE TSE* (1), 157–171 (1986)
24. Sun, S.X., Zhao, J.L., Nunamaker, J.F., Sheng, O.R.L.: Formulating the data-flow perspective for business process management. *Inf. Syst. Research* **17**(4) (2006)
25. Temporal Technologies: Temporal: Durable execution and workflow orchestration. <https://temporal.io/> (2026)
26. Trcka, N., van der Aalst, W.M.P., Sidorova, N.: Data-flow anti-patterns: Discovering data-flow errors in workflows. In: *CAiSE. LNCS*, vol. 5565, pp. 425–439 (2009)
27. Verbeek, H.M.W., Basten, T., van der Aalst, W.M.P.: Diagnosing workflow processes using Woflan. *The Computer Journal* **44**(4), 246–279 (2001)
28. Zhang, H., Cardoza, A., Chen, P.B., Angel, S., Liu, V.: Fault-tolerant and transactional stateful serverless workflows. In: *OSDI* (2020)